

DDPM — The Complete Mathematics

From First Principles to the Final Training Loss

Sri Karthikeya Annavarjula

Contents

1	Setup and Notation	3
2	The Forward (Noising) Process	3
2.1	One Single Step	3
2.2	Why This Specific Formula Keeps Total Variance Equal to 1	4
2.3	The Full Chain, and Why x_T Becomes Pure Noise	4
3	The “Nice Property”: Jumping to Any Timestep Directly	4
3.1	The Problem We’re Solving	4
3.2	Step-by-Step Derivation	5
4	Why We Need a Lower Bound at All	6
4.1	Step 1 — Multiply by 1	6
4.2	Step 2 — Jensen’s Inequality	6
5	The Joint Distributions (Markov Chain Structure)	7
5.1	The Generative (Reverse) Joint	7
5.2	The Inference (Forward) Joint	7
6	Substituting In — and Why the Naive Result is Broken	7
7	The Fix: Bayes’ Rule + Telescoping Cancellation	8
7.1	Step 1 — Re-condition on x_0 using the Markov property	8
7.2	Step 2 — Flip the direction with Bayes’ Rule	8
7.3	Step 3 — Substitute and Telescope	8
8	The Final General ELBO	9
8.1	Step 1 — Substitute the Bayes-Decomposed Denominator	9
8.2	Step 2 — Group into Three Distinct Fractions	9
8.3	Step 3 — Expand the Logarithm and Flip Log-Ratios	9
8.4	Step 4 — Distribute Expectation and Marginalize Latents	10
8.5	The Final General ELBO Theorem	10

9 The Gaussian KL Divergence Formula	10
9.1 Setting Up	11
9.2 Step-by-step Derivation	11
10 Proof: $\mathbb{E}_q[(x - \mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$	11
11 Closed Form: $q(x_{t-1} \mid x_t, x_0)$	12
12 Reparameterization: Predict the Noise, Not the Mean	13
12.1 Rearranging the Nice Property	13
12.2 Substitute into $\tilde{\mu}_t$	13
12.3 Defining the Decoder's Mean This Way	13
13 The Final Simplified Loss	13
14 Proof: MSE-Optimal Prediction Is the Conditional Mean	14
14.1 Setup	14
14.2 The Add-and-Subtract Trick, Again	14
14.3 Minimizing — No Calculus Needed	15
15 Why Predicting Mean Noise Equals Predicting Mean Image	15
15.1 Step 1 — Solve the Nice Property for ϵ	15
15.2 Step 2 — Take Conditional Expectation of Both Sides	15
15.3 Step 3 — Substitute the Optimal Prediction Back	15
16 Training and Sampling Algorithms	16
16.1 Training (repeat until convergence)	16
16.2 Sampling / Generation (after training)	16
17 Full Summary Cheat-Sheet	17

1 Setup and Notation

Before touching any equations, let's agree on what every symbol means. This is the single most important section to get comfortable with — everything after this is just algebra applied to these definitions.

The cast of characters:

- x_0 — a real image from our dataset (a photo of a cat, a face, whatever). This is the data we actually have.
- $x_1, x_2, \dots, x_T$ — a sequence of increasingly noisy versions of x_0 . Same width $\times$ height $\times$ channels as x_0 — just progressively more corrupted by noise.
- T — total number of noising steps (in real models, often $T = 1000$).
- t — an index telling you *which* step of the chain you're looking at ($t = 0$ is the clean image, $t = T$ is pure noise).
- ϵ — a sample of pure Gaussian noise, i.e. $\epsilon \sim \mathcal{N}(0, I)$ (mean zero, variance one, independent at every pixel).
- α_t — a fixed number between 0 and 1, chosen ahead of time (not learned), controlling how much signal survives at step t .
- $\bar{\alpha}_t := \alpha_1 \alpha_2 \dots \alpha_t$ — the running product of all α 's up to step t .
- θ — the learnable weights of our neural network.

Plain-English Intuition

Think of $x_0, x_1, \dots, x_T$ as frames of a video where a photograph slowly dissolves into TV static. x_0 is the first frame (crisp photo), x_T is the last frame (pure snow/static). Everything in this document is about precisely describing that dissolving process, and then learning to run the video backwards.

2 The Forward (Noising) Process

2.1 One Single Step

We define, by hand, a simple rule for how to go from x_{t-1} to x_t : shrink the image slightly, then add a small amount of fresh Gaussian noise.

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I\right)$$

Using the **reparameterization trick** (turning a random sampling step into a deterministic formula plus separate injected noise, so later we can differentiate through it), this is equivalent to writing:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, I)$$

Plain-English Intuition

Why shrink by $\sqrt{\alpha_t}$ instead of just adding noise on top of the untouched image? If you only ever *add* noise without ever shrinking the signal, the total energy (variance) of the pixel values would keep growing forever — after 1000 additions, your numbers would be astronomically large and nothing like a real image. The shrink factor is tuned so that, at every single step, "a little bit of old signal" is traded for "a little bit of static," keeping the total variance balanced and well-behaved. We will prove exactly how this balance works in the next section.

2.2 Why This Specific Formula Keeps Total Variance Equal to 1

Let's check: if x_{t-1} has variance 1 (per pixel) and mean 0, does x_t also have variance 1? This is important — we want every x_t to live on the same "scale," otherwise the network would need to handle wildly different numeric ranges at every timestep.

$$\text{Var}(x_t) = \text{Var}\left(\sqrt{\alpha_t} x_{t-1}\right) + \text{Var}\left(\sqrt{1 - \alpha_t} \epsilon_{t-1}\right)$$

(We can add variances because x_{t-1} and ϵ_{t-1} are independent — the noise added at this step has nothing to do with the image's own randomness.)

$$= \alpha_t \cdot \text{Var}(x_{t-1}) + (1 - \alpha_t) \cdot \text{Var}(\epsilon_{t-1}) = \alpha_t \cdot 1 + (1 - \alpha_t) \cdot 1 = 1$$

Result: if x_{t-1} has unit variance, so does x_t . By induction, every single x_t in the whole chain has unit variance, regardless of t . This is exactly why the $\sqrt{\alpha_t}$ and $\sqrt{1 - \alpha_t}$ coefficients were chosen this way — it's engineered, not arbitrary.

2.3 The Full Chain, and Why x_T Becomes Pure Noise

Repeating the one-step rule T times:

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} x_2 \cdots \rightarrow x_{T-1} \xrightarrow{q(x_T|x_{T-1})} x_T$$

The schedule $\alpha_1, \alpha_2, \dots, \alpha_T$ is chosen (this is called the **noise schedule** — e.g. linearly decreasing, or a cosine schedule) so that $\bar{\alpha}_T = \alpha_1 \alpha_2 \cdots \alpha_T \approx 0$. As we will see in the next section, this guarantees that whatever x_0 we started with, x_T ends up statistically indistinguishable from $\mathcal{N}(0, I)$ — pure noise, with zero memory of the original image.

3 The “Nice Property”: Jumping to Any Timestep Directly

This is the single most important shortcut in the entire theory. Without it, training would be far too slow to be practical.

3.1 The Problem We're Solving

To train the network to reverse step $t = 500$, we would (naively) need to run 500 sequential noising steps just to build one training example. Repeat this for millions of training examples, at every possible timestep — training would be crushingly slow.

Plain-English Intuition

We want a shortcut: given only the original clean image x_0 and a target timestep t , produce x_t directly, in one line, with no loop.

3.2 Step-by-Step Derivation

Step 1 — Write the one-step formula.

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}$$

Step 2 — Unroll one level. Substitute the same rule for x_{t-1} in terms of x_{t-2} :

$$x_{t-1} = \sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \epsilon_{t-2}$$

Plug this into the equation for x_t :

$$\begin{aligned} x_t &= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \epsilon_{t-2} \right) + \sqrt{1 - \alpha_t} \epsilon_{t-1} \\ x_t &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \underbrace{\sqrt{\alpha_t (1 - \alpha_{t-1})} \epsilon_{t-2} + \sqrt{1 - \alpha_t} \epsilon_{t-1}}_{\text{two independent noise terms added together}} \end{aligned}$$

Step 3 — Merge the two noise terms. We use a basic probability fact:

Fact: If $A \sim \mathcal{N}(0, a^2 I)$ and $B \sim \mathcal{N}(0, b^2 I)$ are *independent*, then $A + B \sim \mathcal{N}(0, (a^2 + b^2) I)$. Variances of independent Gaussians simply *add* when you sum the random variables.

Here $a^2 = \alpha_t(1 - \alpha_{t-1})$ and $b^2 = 1 - \alpha_t$, so the combined noise has variance:

$$\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = \alpha_t - \alpha_t \alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t \alpha_{t-1}$$

So we can replace both separate noise terms with one single fresh noise variable $\bar{\epsilon} \sim \mathcal{N}(0, I)$:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\epsilon}$$

Step 4 — Notice the pattern and induct. This has *exactly* the same shape as the original one-step formula, just with α_t replaced by the product $\alpha_t \alpha_{t-1}$. Repeating this merge all the way down to x_0 , every pair of neighbouring steps collapses the same way, giving:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad \bar{\alpha}_t := \prod_{s=1}^t \alpha_s$$

Equivalently, as a distribution:

$$q(x_t | x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I\right)$$

Plain-English Intuition

This is a genuine, exact, one-line shortcut — not an approximation. Want a noisy version of a real image at noise level 500? Don't loop 500 times. Just take the real image, shrink it by one precomputed number $\sqrt{\bar{\alpha}_{500}}$, and add one dose of noise scaled by another precomputed number $\sqrt{1 - \bar{\alpha}_{500}}$. Since $\bar{\alpha}_T \approx 0$ by design of the schedule, $x_T \approx \sqrt{0} \cdot x_0 + \sqrt{1} \cdot \epsilon = \epsilon$ — pure noise, confirming what we claimed in Section 2.3.

4 Why We Need a Lower Bound at All

Our real goal is to make the model assign high probability to real images — i.e., maximize $\log p_\theta(x_0)$ over our whole dataset. But this quantity is defined as an integral over *every possible* noise chain that could have produced x_0 :

$$p_\theta(x_0) = \int p_\theta(x_0, x_{1:T}) dx_{1:T}$$

This integral is over an enormous, high-dimensional space and cannot be computed directly. So instead, we derive a **lower bound** that we *can* compute — and train the model to push that lower bound up, which drags the true (uncomputable) quantity up with it.

4.1 Step 1 — Multiply by 1

Multiply inside the integral by $\frac{q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} = 1$ (legal, changes nothing):

$$p_\theta(x_0) = \int p_\theta(x_0, x_{1:T}) \cdot \frac{q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} dx_{1:T}$$

Regroup this as an expectation (recall: $\int q(z)f(z) dz$ is, *by definition*, $\mathbb{E}_q[f(z)]$):

$$p_\theta(x_0) = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right]$$

4.2 Step 2 — Jensen’s Inequality

The log function is **concave** (curves downward, like a frown). For any concave function f and random variable X :

$$f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)] \quad (\text{Jensen’s Inequality})$$

Plain-English Intuition

Quick sanity check with $f(x) = \sqrt{x}$ on the numbers 1 and 9: $\sqrt{\frac{1+9}{2}} = \sqrt{5} \approx 2.24$, but $\frac{\sqrt{1}+\sqrt{9}}{2} = \frac{1+3}{2} = 2$. Averaging first, then applying the curved-down function, gives a bigger number than applying the function first and then averaging. That is the entire content of Jensen’s Inequality.

Apply this with $X = \frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)}$:

$$\log p_\theta(x_0) = \log \mathbb{E}_q \left[\frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right] \geq \mathbb{E}_q \left[\log \frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right]$$

This right-hand side is the ELBO (Evidence Lower BOUND):

$$\text{ELBO} := \mathbb{E}_q \left[\log \frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right] \quad \text{and we have proven} \quad \log p_\theta(x_0) \geq \text{ELBO}$$

This derivation is completely general — it works identically whether we have 1 latent (plain VAE), 3 latents (a small hierarchical VAE), or 1000 latents (DDPM). What changes next is *how* the joint distributions factor.

5 The Joint Distributions (Markov Chain Structure)

5.1 The Generative (Reverse) Joint

We define the reverse process as a Markov chain, starting from pure noise:

$$p_{\theta}(x_{0:T}) = p(x_T) \cdot \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$$

where $p(x_T) = \mathcal{N}(0, I)$ (the prior we sample from at generation time), and each step is Gaussian:

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_t)$$

5.2 The Inference (Forward) Joint

From Section 2, using the Markov property (each step only depends on the immediately preceding one):

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

Plain-English Intuition

“Markov” means each arrow only looks one step backward — x_{57} only cares about x_{56} , never x_{40} or x_0 directly. This restriction is what keeps the whole derivation tractable; without it, every step would need to consider every other step simultaneously, and the math would explode into an unmanageable mess.

6 Substituting In — and Why the Naive Result is Broken

Plugging both joints into the ELBO from Section 4:

$$\text{ELBO} = \mathbb{E}_q \left[\log \frac{p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)}{\prod_{t=1}^T q(x_t | x_{t-1})} \right]$$

If you try to pair up the numerator and denominator term-by-term, you’d compare $p_{\theta}(x_{t-1} | x_t)$ against $q(x_t | x_{t-1})$. Look closely: **these point in opposite directions**. One asks “given x_t , what’s x_{t-1} ?” The other asks “given x_{t-1} , what’s x_t ?” You cannot form a clean, low-variance KL divergence between two distributions that don’t share the same conditioning direction. We need a smarter move.

7 The Fix: Bayes' Rule + Telescoping Cancellation

7.1 Step 1 — Re-condition on x_0 using the Markov property

Take any forward term $q(x_t|x_{t-1})$. Because the chain is Markov, once you know x_{t-1} , knowing x_0 on top gives *zero* extra information about x_t :

$$q(x_t | x_{t-1}) = q(x_t | x_{t-1}, x_0)$$

7.2 Step 2 — Flip the direction with Bayes' Rule

Recall Bayes' Rule: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$. Apply it here (conditioning everything additionally on x_0 throughout):

$$q(x_t | x_{t-1}, x_0) = \frac{q(x_{t-1} | x_t, x_0) \cdot q(x_t | x_0)}{q(x_{t-1} | x_0)}$$

Now the left side of the numerator, $q(x_{t-1}|x_t, x_0)$, points in the *same* direction as our generative term $p_\theta(x_{t-1}|x_t)$ — both take x_t and produce x_{t-1} . They are now directly comparable.

7.3 Step 3 — Substitute and Telescope

Substituting this into the full product $\prod_{t=1}^T q(x_t|x_{t-1})$ (keeping the $t = 1$ term separate, since it conditions on x_0 directly already):

$$\prod_{t=2}^T q(x_t|x_{t-1}) = \prod_{t=2}^T \frac{q(x_{t-1}|x_t, x_0) \cdot q(x_t|x_0)}{q(x_{t-1}|x_0)}$$

Split into two products, and look closely at the second one written out term by term:

$$\frac{q(x_2|x_0)}{q(x_1|x_0)} \times \frac{q(x_3|x_0)}{q(x_2|x_0)} \times \frac{q(x_4|x_0)}{q(x_3|x_0)} \times \dots \times \frac{q(x_T|x_0)}{q(x_{T-1}|x_0)}$$

This is a **telescoping product** — every numerator cancels the next denominator, like a chain of dominoes. Everything collapses to just:

$$\frac{q(x_T|x_0)}{q(x_1|x_0)}$$

Including the standalone $t = 1$ term $q(x_1|x_0)$ that we set aside earlier, the whole denominator becomes:

$$\prod_{t=1}^T q(x_t|x_{t-1}) = q(x_1|x_0) \times \left[\prod_{t=2}^T q(x_{t-1}|x_t, x_0) \right] \times \frac{q(x_T|x_0)}{q(x_1|x_0)} = q(x_T|x_0) \times \prod_{t=2}^T q(x_{t-1}|x_t, x_0)$$

The $q(x_1|x_0)$ cancels itself out entirely. Clean.

8 The Final General ELBO

Now we explicitly substitute our telescoped Bayes result from Section 7.3 back into the general ELBO expectation from Section 6. We unroll every step of the algebra below to make every substitution, logarithm property, and expectation marginalization crystal clear.

8.1 Step 1 — Substitute the Bayes-Decomposed Denominator

Recall the ELBO from Section 6:

$$\text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)}{\prod_{t=1}^T q(x_t | x_{t-1})} \right]$$

From Section 7.3, the product of forward terms in the denominator simplified to:

$$\prod_{t=1}^T q(x_t | x_{t-1}) = q(x_T | x_0) \cdot \prod_{t=2}^T q(x_{t-1} | x_t, x_0)$$

Plugging this denominator into the ELBO, and separating out the $t = 1$ term $p_\theta(x_0 | x_1)$ from the numerator product:

$$\text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \left(\frac{p_\theta(x_0 | x_1) \cdot p(x_T) \cdot \prod_{t=2}^T p_\theta(x_{t-1} | x_t)}{q(x_T | x_0) \cdot \prod_{t=2}^T q(x_{t-1} | x_t, x_0)} \right) \right]$$

8.2 Step 2 — Group into Three Distinct Fractions

Group corresponding terms inside the fraction:

$$\text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \left(p_\theta(x_0 | x_1) \times \frac{p(x_T)}{q(x_T | x_0)} \times \prod_{t=2}^T \frac{p_\theta(x_{t-1} | x_t)}{q(x_{t-1} | x_t, x_0)} \right) \right]$$

8.3 Step 3 — Expand the Logarithm and Flip Log-Ratios

Using the product rule for logarithms ($\log(A \cdot B \cdot C) = \log A + \log B + \log C$), the expression splits into three terms:

$$\text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log p_\theta(x_0 | x_1) + \log \frac{p(x_T)}{q(x_T | x_0)} + \sum_{t=2}^T \log \frac{p_\theta(x_{t-1} | x_t)}{q(x_{t-1} | x_t, x_0)} \right]$$

To express the second and third terms as standard KL divergences (which have q in the numerator of the logarithm), we use $\log \frac{A}{B} = -\log \frac{B}{A}$:

$$\begin{aligned} \log \frac{p(x_T)}{q(x_T | x_0)} &= -\log \frac{q(x_T | x_0)}{p(x_T)} \\ \log \frac{p_\theta(x_{t-1} | x_t)}{q(x_{t-1} | x_t, x_0)} &= -\log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} \end{aligned}$$

Substituting these flipped log-ratios back into the expectation:

$$\text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log p_\theta(x_0 | x_1) - \log \frac{q(x_T | x_0)}{p(x_T)} - \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} \right]$$

8.4 Step 4 — Distribute Expectation and Marginalize Latents

By linearity of expectation, we distribute $\mathbb{E}_{q(x_{1:T}|x_0)}$ term-by-term and marginalize out variables that do not appear in each term:

1. **Reconstruction Term:** $\log p_\theta(x_0 | x_1)$ only depends on x_1 . Integrating out $x_2, \dots, x_T$ reduces the joint expectation $\mathbb{E}_{q(x_{1:T}|x_0)}$ to just $\mathbb{E}_{q(x_1|x_0)}$:

$$\mathbb{E}_{q(x_{1:T}|x_0)} [\log p_\theta(x_0 | x_1)] = \mathbb{E}_{q(x_1|x_0)} [\log p_\theta(x_0 | x_1)]$$

2. **Prior Matching Term:** $\log \frac{q(x_T|x_0)}{p(x_T)}$ depends only on x_T . Integrating out $x_1, \dots, x_{T-1}$ reduces the expectation to $\mathbb{E}_{q(x_T|x_0)}$. By definition of KL divergence ($D_{KL}(q||p) := \mathbb{E}_q[\log \frac{q}{p}]$):

$$-\mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_T | x_0)}{p(x_T)} \right] = -\mathbb{E}_{q(x_T|x_0)} \left[\log \frac{q(x_T | x_0)}{p(x_T)} \right] = -D_{KL}(q(x_T | x_0) || p(x_T))$$

3. **Denoising Matching Terms:** Each term in the sum depends only on x_t and x_{t-1} . Marginalizing out all other variables yields:

$$\begin{aligned} & - \sum_{t=2}^T \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} \right] \\ &= - \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} \left[\mathbb{E}_{q(x_{t-1}|x_t, x_0)} \left[\log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} \right] \right] \\ &= - \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} \left[D_{KL}(q(x_{t-1} | x_t, x_0) || p_\theta(x_{t-1} | x_t)) \right] \end{aligned}$$

8.5 The Final General ELBO Theorem

Putting all three marginalized terms together gives the final, full mathematical expression for the ELBO:

$$\begin{aligned} \log p_\theta(x_0) &\geq \underbrace{\mathbb{E}_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)]}_{\text{(A) Reconstruction}} - \underbrace{D_{KL}(q(x_T|x_0) || p(x_T))}_{\text{(B) Prior matching}} \\ &\quad - \sum_{t=2}^T \underbrace{\mathbb{E}_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))]}_{\text{(C) Denoising match, one per step}} \end{aligned}$$

Term	Plain-English meaning
(A) Reconstruction	Given the first latent x_1 , can you rebuild the real image x_0 ?
(B) Prior matching	Does x_T (walking all the way up from x_0) actually look like clean noise $\mathcal{N}(0, I)$?
(C) Denoising match	At each step t , does the decoder's guess for x_{t-1} match the best-possible (Bayes-computed) guess, given that we secretly know the true x_0 ?

9 The Gaussian KL Divergence Formula

Every KL term above is between two Gaussians. We need a closed-form formula so we never have to estimate these terms via sampling.

9.1 Setting Up

Let $q = \mathcal{N}(\mu_1, \sigma_1^2)$ and $p = \mathcal{N}(\mu_2, \sigma_2^2)$ (1-D case; the multi-D case is identical, just with vectors). By definition:

$$D_{KL}(q\|p) = \mathbb{E}_q[\log q(x) - \log p(x)]$$

9.2 Step-by-step Derivation

Step 1 — write out the log-Gaussian densities:

$$\log q(x) = -\frac{1}{2} \log(2\pi\sigma_1^2) - \frac{(x - \mu_1)^2}{2\sigma_1^2}, \quad \log p(x) = -\frac{1}{2} \log(2\pi\sigma_2^2) - \frac{(x - \mu_2)^2}{2\sigma_2^2}$$

Step 2 — subtract:

$$\log q(x) - \log p(x) = \frac{1}{2} \log \frac{\sigma_2^2}{\sigma_1^2} - \frac{(x - \mu_1)^2}{2\sigma_1^2} + \frac{(x - \mu_2)^2}{2\sigma_2^2}$$

Step 3 — take the expectation under q . We need two facts, both proven below in Section 9:

- $\mathbb{E}_q[(x - \mu_1)^2] = \sigma_1^2$ — this is literally the definition of variance.
- $\mathbb{E}_q[(x - \mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$ — proven fully in Section 9.

Step 4 — substitute:

$$D_{KL}(q\|p) = \frac{1}{2} \log \frac{\sigma_2^2}{\sigma_1^2} - \frac{\sigma_1^2}{2\sigma_1^2} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2}$$

$$D_{KL}(q\|p) = \frac{1}{2} \left[\log \frac{\sigma_2^2}{\sigma_1^2} + \frac{\sigma_1^2}{\sigma_2^2} + \frac{(\mu_1 - \mu_2)^2}{\sigma_2^2} - 1 \right]$$

If the two variances are forced equal ($\sigma_1 = \sigma_2$, a common simplifying design choice), the first three terms collapse and only one piece survives:

$$D_{KL}(q\|p) \propto (\mu_1 - \mu_2)^2$$

Plain-English Intuition

This is huge: comparing two Gaussians with matched variance reduces to plain squared error between their means — no integrals, no sampling, just ordinary MSE.

10 Proof: $\mathbb{E}_q[(x - \mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$

This is the “bias–variance” identity used above. Let’s prove it fully, since it looks surprising at first (σ_2 never appears!).

Step 1 — add and subtract μ_1 inside the square:

$$x - \mu_2 = (x - \mu_1) + (\mu_1 - \mu_2)$$

Check: $(x - \mu_1) + (\mu_1 - \mu_2) = x - \mu_2$. Correct — pure algebra, nothing hidden.

Step 2 — square it. Let $a := x - \mu_1$ (random) and $b := \mu_1 - \mu_2$ (a fixed constant, no randomness):

$$(x - \mu_2)^2 = (a + b)^2 = a^2 + 2ab + b^2$$

Step 3 — take expectations term by term:

$$\mathbb{E}_q[(x - \mu_2)^2] = \mathbb{E}_q[a^2] + 2\mathbb{E}_q[ab] + \mathbb{E}_q[b^2]$$

- **Piece 1:** $\mathbb{E}_q[a^2] = \mathbb{E}_q[(x - \mu_1)^2] = \sigma_1^2$ — a genuine variance, since μ_1 truly is q ’s own mean.
- **Piece 2:** $2\mathbb{E}_q[ab] = 2b \cdot \mathbb{E}_q[x - \mu_1] = 2b \cdot (\mu_1 - \mu_1) = 2b \cdot 0 = 0$. This vanishes precisely because μ_1 is the *true* mean of q — $\mathbb{E}_q[x] = \mu_1$ by definition.
- **Piece 3:** $\mathbb{E}_q[b^2] = (\mu_1 - \mu_2)^2$ — a constant, expectation does nothing to it.

$$\mathbb{E}_q[(x - \mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$$

Plain-English Intuition

Archery analogy: σ_1^2 is how tightly your arrows group around *where you’re actually aiming* (μ_1) — pure spread. $(\mu_1 - \mu_2)^2$ is how far off your aim point is from some *other* target (μ_2) you’re being scored against — pure bias. Mean squared distance from any point = variance around your true mean + squared distance from that true mean to the point. σ_2 never enters because you’re not averaging over p at all — you’re averaging over q .

11 Closed Form: $q(x_{t-1} \mid x_t, x_0)$

We now derive, in exact closed form, the target distribution that term (C) from Section 8 needs the decoder to match. By Bayes’ Rule:

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1}, x_0) q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} = \frac{q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)}$$

(using the Markov property again for the last equality). We already know all three ingredients in closed form:

- $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I)$ — the one-step forward formula (Section 2.1)
- $q(x_{t-1} \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}} x_0, (1 - \bar{\alpha}_{t-1})I)$ — the nice property, one step earlier (Section 3)
- $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$ — the nice property (Section 3)

Multiplying and dividing Gaussian densities always produces another Gaussian (completing the square in the exponent — mechanical but tedious algebra, omitted here for length, but the family of Gaussians is provably closed under this operation). The result:

$$\begin{aligned} q(x_{t-1} | x_t, x_0) &= \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I) \\ \tilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} x_0 \\ \tilde{\beta}_t &= \frac{(1 - \bar{\alpha}_{t-1})(1 - \alpha_t)}{1 - \bar{\alpha}_t} \end{aligned}$$

Plain-English Intuition

Plain English: *if* you already know both the noisy image x_t *and* the true clean image x_0 , there is an exact, closed-form best guess for x_{t-1} — it's just a weighted average of x_t and x_0 . No neural network is needed to compute this side, since during training we really do know x_0 (it's our training data). This is exactly the target the decoder network is being trained to match.

12 Reparameterization: Predict the Noise, Not the Mean

12.1 Rearranging the Nice Property

From Section 3: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$. Solve for x_0 :

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

12.2 Substitute into $\tilde{\mu}_t$

Plugging this into the $\tilde{\mu}_t(x_t, x_0)$ formula from Section 11 and simplifying (mechanical algebra — substitute, collect terms, cancel factors of $\sqrt{\bar{\alpha}_t}$ and $\sqrt{\bar{\alpha}_{t-1}}$):

$$\tilde{\mu}_t = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

Plain-English Intuition

The only *unknown* quantity left in this formula is ϵ — the specific noise that was added. Everything else (x_t , α_t , $\bar{\alpha}_t$) is already known at training time. This is the key realization: instead of training the network to output a mean directly, reparameterize it to predict the noise $\epsilon_\theta(x_t, t)$, and reconstruct the mean using this exact same formula.

12.3 Defining the Decoder's Mean This Way

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

13 The Final Simplified Loss

Plug $\tilde{\mu}_t$ and μ_θ (same functional form!) into the KL/MSE result from Section 8:

$$D_{KL} \propto \|\tilde{\mu}_t - \mu_\theta\|^2 = \left\| \frac{1}{\sqrt{\alpha_t}} \cdot \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} (\epsilon_\theta(x_t, t) - \epsilon) \right\|^2$$

Everything outside the parentheses is just a fixed scalar weight depending on t (not on the network). Dropping this weight (an empirical choice — the authors found training without it produces *better* sample quality, even though it technically departs from the exact ELBO) gives the famous simplified DDPM loss:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} \left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right]$$

Plain-English Intuition

In plain English: at a random timestep t , add known random noise ϵ to a real image to make x_t , then train a network to guess what that noise ϵ was, just from looking at x_t and t . That's it — the entire DDPM training objective, no KL formulas needed at training time, just a plain squared-error regression loss.

14 Proof: MSE-Optimal Prediction Is the Conditional Mean

Why does minimizing $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$ force ϵ_θ toward $\mathbb{E}[\epsilon \mid x_t]$? Let's prove this rigorously with pure algebra (no calculus needed).

14.1 Setup

Fix a specific x_t . Define $m := \mathbb{E}[\epsilon \mid x_t]$ — the true, fixed conditional mean noise (determined by the real data distribution). The network picks some output $g := \epsilon_\theta(x_t, t)$ to minimize:

$$L(g) = \mathbb{E}_{\epsilon|x_t} [(\epsilon - g)^2]$$

14.2 The Add-and-Subtract Trick, Again

$$\epsilon - g = (\epsilon - m) + (m - g)$$

$$(\epsilon - g)^2 = (\epsilon - m)^2 + 2(\epsilon - m)(m - g) + (m - g)^2$$

Take expectations (note $(m - g)$ is a constant w.r.t. ϵ , so it factors out of the middle term):

$$L(g) = \mathbb{E}[(\epsilon - m)^2] + 2(m - g) \underbrace{\mathbb{E}[\epsilon - m]}_{=0, \text{ since } m=\mathbb{E}[\epsilon]} + (m - g)^2$$

$$L(g) = \underbrace{\mathbb{E}[(\epsilon - m)^2]}_{\text{fixed constant, independent of } g} + (m - g)^2$$

14.3 Minimizing — No Calculus Needed

The first term is fixed, no matter what g is. The second term, $(m - g)^2$, is always ≥ 0 , and equals *exactly zero* if and only if $g = m$. So:

$$L(g) \geq (\text{fixed constant}), \quad \text{with equality iff } g = m$$

$$g^* = \arg \min_g L(g) = m = \mathbb{E}[\epsilon \mid x_t]$$

This extends immediately to the full image (a vector, not a scalar): since $\|\epsilon - \hat{\epsilon}\|^2 = \sum_i (\epsilon_i - \hat{\epsilon}_i)^2$ splits into independent per-pixel terms, the exact same proof applies at every pixel simultaneously, giving $\hat{\epsilon}_i^* = \mathbb{E}[\epsilon_i \mid x_t]$ for every pixel i .

15 Why Predicting Mean Noise Equals Predicting Mean Image

A natural worry: the network only ever outputs *noise* predictions — how do we know the *image* you reconstruct from it is also an average (and therefore also potentially blurry at high ambiguity)? Let's prove the connection is *exact*, not approximate.

15.1 Step 1 — Solve the Nice Property for ϵ

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \implies \epsilon = \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}$$

15.2 Step 2 — Take Conditional Expectation of Both Sides

Given x_t (now a fixed, known quantity — not random), only x_0 remains random on the right. Using linearity of expectation ($\mathbb{E}[a - bY] = a - b\mathbb{E}[Y]$ for constants a, b):

$$\mathbb{E}[\epsilon \mid x_t] = \frac{x_t - \sqrt{\bar{\alpha}_t}\mathbb{E}[x_0 \mid x_t]}{\sqrt{1 - \bar{\alpha}_t}}$$

This is an *exact algebraic identity*: average noise given x_t is a simple linear function of average clean image given x_t .

15.3 Step 3 — Substitute the Optimal Prediction Back

Recall the reconstruction formula: $\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}$. Plug in the MSE-optimal $\hat{\epsilon} = \mathbb{E}[\epsilon \mid x_t]$ from Section 13, using the identity just derived:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}\mathbb{E}[x_0 \mid x_t]}{\sqrt{1 - \bar{\alpha}_t}}}{\sqrt{\bar{\alpha}_t}} = \frac{x_t - (x_t - \sqrt{\bar{\alpha}_t}\mathbb{E}[x_0 \mid x_t])}{\sqrt{\bar{\alpha}_t}} = \frac{\sqrt{\bar{\alpha}_t}\mathbb{E}[x_0 \mid x_t]}{\sqrt{\bar{\alpha}_t}}$$

$$\hat{x}_0 = \mathbb{E}[x_0 \mid x_t]$$

Plain-English Intuition

When the network achieves its MSE-optimal noise prediction, algebraically converting it back into an image gives you *exactly* the true conditional average of all plausible clean images given x_t — not approximately, by clean algebra. This works because the $\epsilon \leftrightarrow x_0$ relationship is exactly linear, and expectation commutes cleanly through linear transformations. This is precisely why one-shot jumps from pure noise to a finished image are blurry (huge ambiguity being averaged over), while small steps stay sharp (tiny ambiguity, barely any blur).

16 Training and Sampling Algorithms

16.1 Training (repeat until convergence)

1. Sample a real image: $x_0 \sim \text{data}$
2. Sample a random timestep: $t \sim \text{Uniform}(1, \dots, T)$
3. Sample noise: $\epsilon \sim \mathcal{N}(0, I)$
4. Build x_t directly using the nice property: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. Compute loss: $\mathcal{L} = \|\epsilon - \epsilon_\theta(x_t, t)\|^2$
6. Backpropagate and update θ with one gradient step

16.2 Sampling / Generation (after training)

1. Start with pure noise: $x_T \sim \mathcal{N}(0, I)$
2. For $t = T, T - 1, \dots, 1$:
 - (a) Predict noise: $\hat{\epsilon} = \epsilon_\theta(x_t, t)$
 - (b) Compute the mean: $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right)$
 - (c) Sample: $x_{t-1} \sim \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t)$ (inject fresh randomness, except at the very last step)
3. Output x_0 — the final generated image

Plain-English Intuition

Why re-inject fresh noise at every sampling step instead of always taking the mean deterministically? If you always took the literal mean, you'd repeatedly commit to the “safest, most generic average” at every step, sliding toward a bland, blurry compromise. Injecting a controlled amount of fresh randomness lets the trajectory gradually commit to one specific, sharp, plausible realization instead.

17 Full Summary Cheat-Sheet

Concept	Key Formula
One-step forward	$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}$
Nice property (shortcut)	$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ $\log p(x_0) \geq \mathbb{E}[\log p_\theta(x_0 x_1)] - D_{KL}(q(x_T x_0) p(x_T))$
ELBO (general)	$- \sum_{t \geq 2} \mathbb{E}[D_{KL}(q(x_{t-1} x_t, x_0) p_\theta(x_{t-1} x_t))]$
Gaussian KL	$D_{KL} = \frac{1}{2} \left[\log \frac{\sigma_2^2}{\sigma_1^2} + \frac{\sigma_1^2}{\sigma_2^2} + \frac{(\mu_1 - \mu_2)^2}{\sigma_2^2} - 1 \right]$
True posterior mean	$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$
Final training loss	$\mathcal{L}_{\text{simple}} = \mathbb{E}[\ \epsilon - \epsilon_\theta(x_t, t)\ ^2]$
MSE-optimal prediction	$\epsilon_\theta^*(x_t, t) = \mathbb{E}[\epsilon x_t]$

The whole story in one sentence: destroying an image with small, known Gaussian noise steps is trivial to define exactly and collapses into one closed-form jump-to-any-timestep formula; reversing that destruction is intractable in one shot, but tractable one small step at a time; the exact per-step target has a closed Gaussian form thanks to Bayes' rule and telescoping cancellation; reparameterizing that target in terms of the injected noise turns the whole training objective into an ordinary, stable squared-error regression — the single simplest loss function in all of deep learning, yet powerful enough to generate photorealistic images.